

QSPI FLASH CONTROLLER WITH EXECUTE-IN-PLACE (XIP)

Design Specification

Indirect APB4 Path | Memory-Mapped AHB-Lite XIP Path

By,
Sethumadhavan Bharathi
Emp Id: 6931

1. Overview

1.1 What this controller is

This is a generic QSPI transfer engine, not a command-set implementation.

The hardware contains no per-command knowledge. It does not decode opcodes. It does not know that 0x20 is an erase or that 0x02 is a program. Firmware writes a command descriptor into registers — opcode, address, per-phase line width, dummy-cycle count, direction and length — and the engine clocks out exactly the phase sequence that descriptor specifies.

The consequence is that almost no command needs dedicated RTL. Any JEDEC-compatible NOR flash using 3-byte addressing and SPI mode 0 is supported by writing different register values, not by changing hardware.

1.2 The phase pipeline

Every command is built from the same five-phase pipeline. Each phase is individually enabled and individually width-selectable at 1, 2 or 4 lines.

opcode -> address (3 or 4 bytes) -> mode byte -> dummy (0-31) -> data

Get this engine right and every command in the device's instruction set becomes reachable from firmware without further hardware work.

1.3 Two access paths

Path	Interface	Purpose
Indirect	APB4 slave	Firmware programs a descriptor and starts it. Any command is reachable: programming, erase, status, read.
XIP	AHB-Lite slave	A CPU read to the mapped range is translated automatically into a Quad I/O Read. Read only.

Both paths drive the SAME command engine. Qspi_Core, Qspi_Fsm, Tx_Shift_Reg and Rx_Shift_Reg are shared and are not duplicated. XIP is implemented as a second source of the command descriptor plus arbitration, not as a second datapath.

Only one command is hard-wired in the entire design: Quad I/O Read (0xEB), used by XIP. That is unavoidable, because in XIP mode there is no firmware in the loop to supply a descriptor.

2. Architecture

2.1 Module hierarchy

Module	Domain	Function
qspi_Controller	Top	Integration, FIFO steering, IO pads
Apb_Slave	PCLK	APB4 protocol, address and strobe decode, PREADY
Register_Bank	PCLK	Descriptor registers, TX/RX data staging
Fifo_Handler	PCLK	32-bit APB word to byte-stream packing
CDC_Block	PCLK/SCLK	Descriptor crossing (handshake + hold registers)
Handshake_Sync	PCLK	Descriptor transmitter
Handshake_Reciver	SCLK	Descriptor receiver
Ahb_Xip_Slave	HCLK	AHB-Lite protocol, decode, wait states, errors
Xip_Prefetch_Buffer	HCLK	Line cache, hit/miss, line fill, invalidation
Xip_Cdc	HCLK/SCLK	XIP request crossing
Xip_Handshake_Sync	HCLK	XIP request transmitter
Xip_Handshake_Receiver	SCLK	XIP request receiver
Xip_Arbiter	SCLK	Engine ownership, received-byte counting
Xip_Coherency	SCLK/HCLK	Prefetch invalidate generation
Xip_Descriptor_Mux	SCLK	Indirect vs XIP descriptor selection
Qspi_Core	SCLK	FSM and shift-register integration
Qspi_Fsm	SCLK	Phase sequencer
Tx_Shift_Reg	SCLK	Serialiser, 1/2/4-line
Rx_Shift_Reg	SCLK	Deserialiser, 1/2/4-line
Async_FIFO	cross	Gray-pointer asynchronous FIFO
Sync	any	2-flop synchroniser, parameterised width
Reset_Sync	any	Async assert, sync de-assert reset

2.2 Top-Level Interface Signals

Signal	Direction	Width	Description
PCLK	Input	1	APB clock for the indirect access path.
PRESETn	Input	1	Active-low asynchronous reset for the APB domain.
HCLK	Input	1	AHB-Lite/XIP clock for the memory-mapped access path.
HRESETn	Input	1	Active-low asynchronous reset for the AHB/XIP domain.
SCLK	Input	1	QSPI command-engine clock used by the QSPI core, FSM and shift registers.
SRESETn	Input	1	Active-low asynchronous reset for the QSPI engine domain.
PSEL	Input	1	APB peripheral select signal.
PENABLE	Input	1	APB enable signal indicating the access phase.
PWRITE	Input	1	APB transfer direction: write when high, read when low.
PSTRB	Input	4	APB byte-write strobes used for register and DATA accesses.
PADDR	Input	8	APB register address.
PWDATA	Input	32	APB write data.
PRDATA	Output	32	APB read data returned to the processor.
PREADY	Output	1	APB transfer completion signal. Low inserts wait states; high completes the transfer.

PSLVERR	Output	1	APB error indication for illegal accesses or QSPI command errors.
HSEL	Input	1	AHB-Lite slave select signal.
HADDR	Input	32	AHB-Lite address for XIP memory accesses.
HTRANS	Input	2	AHB-Lite transfer type indication.
HWRITE	Input	1	AHB-Lite transfer direction. XIP supports read accesses only.
HSIZE	Input	3	AHB-Lite transfer size. The XIP path supports word accesses.
HBURST	Input	3	AHB-Lite burst type indication.
HWDATA	Input	32	AHB-Lite write data. Unused because XIP is read-only.
HREADY	Input	1	AHB system ready signal used to qualify the address phase.
HRDATA	Output	32	Data returned to the AHB-Lite master for an XIP read.
HREADYOUT	Output	1	AHB-Lite slave ready signal. Low stalls an access; high completes it.
HRESP	Output	2	AHB-Lite response indication, including OKAY/ERROR responses.
xip_en	Input	1	Enables the XIP access path.
spi_cs_n	Output	1	Active-low chip-select signal connected to the external QSPI flash.
spi_sclk	Output	1	Serial clock supplied to the QSPI flash.
Io[3:0]	Inout	4	Quad-SPI bidirectional data interface connected to flash IO0–IO3.

2.3 Request flow, indirect path

APB write of descriptor registers

- > Register_Bank - Latch opcode, address, widths, length
- > APB write of CONTROL with START
- > CDC_Block - handshake, descriptor crossed to SCLK
- > Xip_Arbiter - grants the engine to the indirect path
- > Xip_Descriptor_Mux - selects the register descriptor
- > Qspi_Fsm - opcode / address / mode / dummy / data
- > flash

2.4 Request flow, XIP path

AHB read to the mapped range

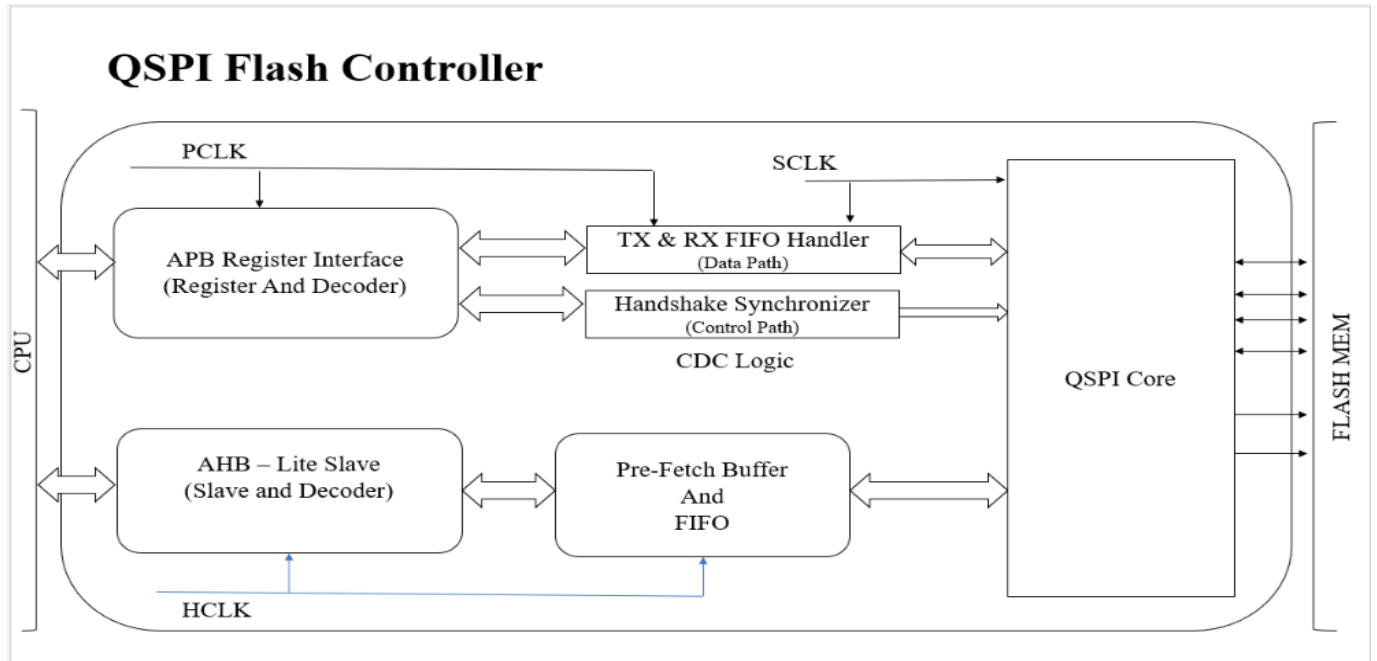
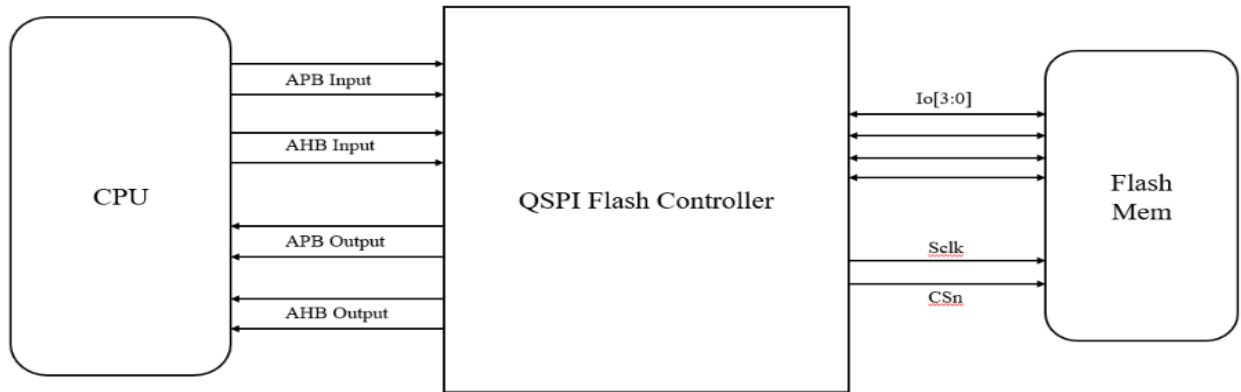
- > Ahb_Xip_Slave - decode, capture address, HREADYOUT low
- > Xip_Prefetch_Buffer - tag compare: hit answers here miss requests a line fill
- > Xip_Cdc - handshake, address and length to SCLK
- > Xip_Arbiter - grants the engine when idle
- > Xip_Descriptor_Mux - selects the fixed 0xEB descriptor
- > Qspi_Fsm - same engine, unchanged
- > flash

2.5 Response flow

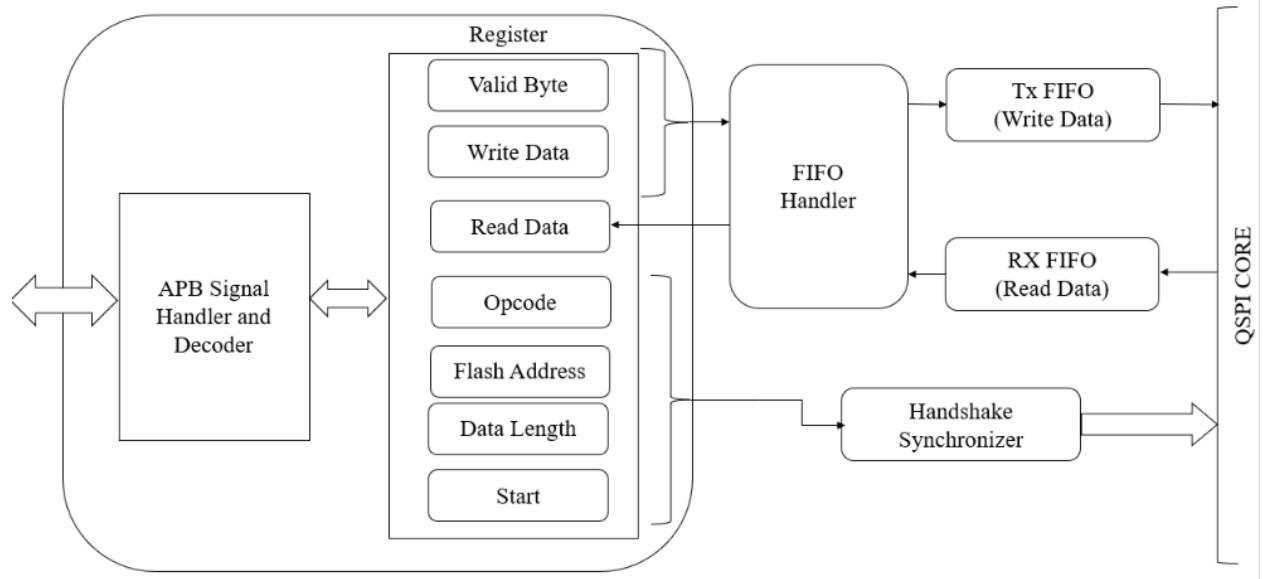
flash

- > Rx_Shift_Reg - assemble bytes, pulse Write_Data
- > RX FIFO - indirect FIFO or XIP FIFO, by ownership
- > Register_Bank - (indirect) firmware pops via DATA
- Xip_Prefetch_Buffer - (XIP) fill the line, select the word
- > Ahb_Xip_Slave - drive HRDATA, raise HREADYOUT

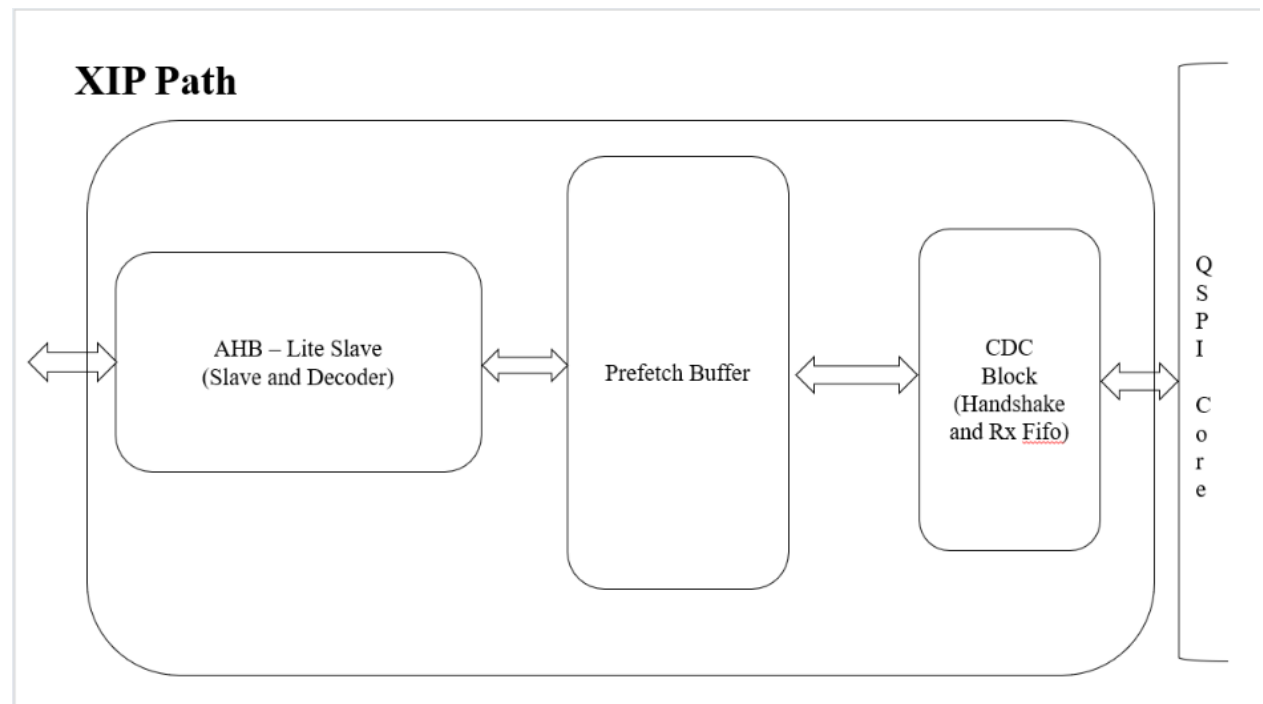
Top-Level Architecture



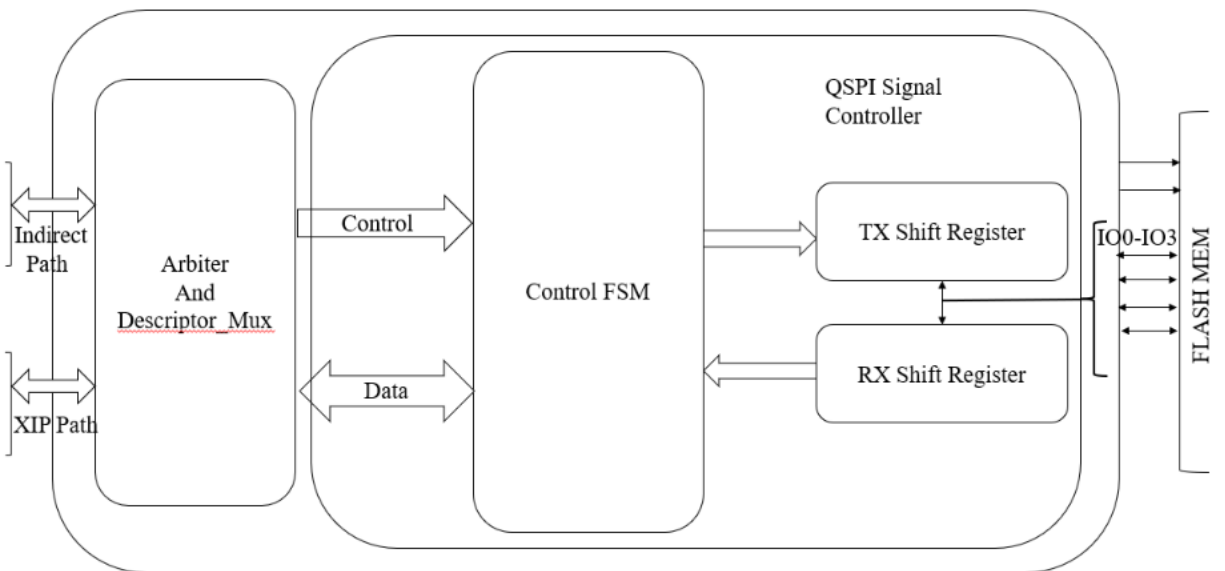
Indirect Path



XIP Path



QSPI Core



3. Clocks and Reset

3.1 Clock domains

Domain	Contains	Verified at
PCLK	APB slave, register bank, indirect FIFO handler	100 MHz
HCLK	AHB slave, prefetch buffer	100 MHz
SCLK	Command engine, shift registers, arbiter	20 MHz

SCLK is fully asynchronous to both PCLK and HCLK. PCLK and HCLK are declared as separate domains in the RTL and may be driven independently; they are tied together in the current integration.

3.2 Reset

All resets are active-low, asynchronously asserted and synchronously de-asserted. A Reset_Sync instance is placed in each domain:

```
Reset_Sync Apb (.clk(PCLK), .reset_n(PRESETn), .reset_sync_n(PRESETn_S));  
Reset_Sync Ahb (.clk(HCLK), .reset_n(HRESETn), .reset_sync_n(HRESETn_S));  
Reset_Sync qspi (.clk(SCLK), .reset_n(SRESETn), .reset_sync_n(SRESETn_S));
```

Every functional block is reset by the synchronised output. The raw reset inputs reach only the synchroniser instances.

3.3 Dedicated XIP receive FIFO

The indirect path and XIP each have their own RX FIFO. Sharing a single FIFO would allow a byte that firmware pushed but never read to be consumed as the first byte of an XIP word, shifting the entire result.

```
rx_fifo_wr_en = write_data && !rx_full    && !xip_owns;  
xip_rx_wr_en  = write_data && !xip_rx_full && xip_owns;
```

Both write sides are in SCLK, where xip_owns is generated, so the steering signal needs no synchroniser.

4. Indirect Path (APB4)

4.1 Register map

PSTRB is checked strictly. A write with a strobe other than the one listed returns PSLVERR and the register is not updated. Any offset not listed returns PSLVERR.

Offset	Name	Access	Required PSTRB	Width
0x04	CONTROL	RW	4'b0111	17
0x08	OPCODE	RW	4'b0001	8
0x0C	ADDRESS	RW	4'b1111	32
0x10	MODE	RW	4'b0001	8
0x14	DUMMY	RW	4'b0001	8
0x18	LENGTH	RW	4'b0001	8
0x1C	DATA	RW	0001 / 0011 / 0111 / 1111	32

4.1.1 CONTROL (0x04)

Bits	Field	Description
0	start	Launches the command. Does not self-clear.
1	opcode_en	Enable the opcode phase
2	addr_en	Enable the address phase
3	mode_en	Enable the mode-byte phase
4	dummy_en	Enable the dummy phase
5	data_w_en	Data phase, write direction
6	data_r_en	Data phase, read direction
8:7	opcode_line	Opcode phase width (see 8.4)
10:9	addr_line	Address phase width
12:11	data_line	Data phase width
13	poll_wip_en	Hardware WIP auto-poll (see 8.2)
15:14	mode_line	Mode-byte phase width
16	addr_4byte	0 = 3-byte address, 1 = 4-byte (see 8.5)

Line-width encoding: 00 = 1-bit, 01 = 2-bit (dual), 10 = 4-bit (quad).

Setting data_w_en and data_r_en together is an illegal descriptor and asserts Error; the command does not run.

4.1.2 DATA (0x1C)

Write: pushes 1 to 4 bytes into the TX FIFO. The byte count is determined by PSTRB, least-significant byte first.

Read: pops one byte from the RX FIFO into PRDATA[7:0].

4.2 Transfer completion and PREADY

PREADY follows standard AMBA polarity: PREADY = 1 completes the transfer, PREADY = 0 inserts a wait state.

Access	PREADY behaviour
Plain register write or read	Asserted the cycle after the access phase begins. Single wait state.
Data read or write	Asserted when the FIFO transaction completes.
CONTROL write with START	Held LOW until the flash command completes. See below.
Illegal address or strobe	Asserted with PSLVERR.

4.2.1 Blocking START

A CONTROL write that asserts START does not complete until the flash command has finished. The APB slave enters a Busy state and holds PREADY low; when the core reports completion, PREADY is asserted and the write completes.

Firmware therefore learns of completion from the write finishing. No status register is required and no polling loop is needed.

Error handling: if the core reports an error, PREADY is asserted together with PSLVERR, so an illegal descriptor is reported through the same mechanism.

CONSEQUENCE: a program or erase stalls the APB bus for the duration of the flash operation, which can be tens of milliseconds. There is no timeout in the Busy path.

4.3 Firmware sequence

1. Write OPCODE, ADDRESS, MODE, DUMMY, LENGTH as the command requires.
2. For a write command, push all data bytes to DATA before starting. There is no underrun stall.
3. Write CONTROL with the phase enables and widths, START = 0.
4. Write CONTROL again with START = 1. This write blocks.
5. When the write completes, the command has finished. Check PSLVERR for an error.
6. Write CONTROL with START = 0.
7. For a read command, pop LENGTH bytes from DATA.

For a program or erase, poll RDSR1 until WIP clears before issuing the next command. The device accepts only status reads while WIP is set.

4.4 Quad mode enable

Quad commands, including the XIP read, require the flash Quad Enable bit to be set first:

WREN (0x06) -> WRSR (0x01) with data 0x40 -> poll RDSR1 until WIP clears

The QE bit location is vendor-specific: Macronix places it in Status Register 1 bit 6, Winbond in Status Register 2 bit 1. Hardware therefore does not program it, which preserves the property that the controller holds no device-specific knowledge.

4.5 Data length and transfer count

LENGTH is 8 bits wide, giving a range of 1 to 255 bytes per indirect command. LENGTH = 0 is guarded and terminates the command immediately. This ceiling is a register-width limit, not a device limit: the flash itself has no maximum burst length and streams data until chip select rises.

The read and write sides move data at different granularities:

- Read: each DATA read pops exactly one byte into PRDATA[7:0] (4.1.2). Reading the maximum 255 bytes therefore takes 255 separate APB read transactions, one per byte.
- Write: PSTRB selects 1 to 4 bytes per DATA write. Pushing 255 bytes takes as few as 64 write transactions — 63 full words with PSTRB = 4'b1111 plus one partial word for the remainder.

The asymmetry is a real gap rather than an inherent limit: the read path could pack up to 4 bytes into PRDATA per transaction the same way the write path already packs on the way in. This would cut the read transaction count by up to 4x and is not implemented; see Future Work.

5. XIP Path (AHB-Lite)

5.1 Purpose

The indirect path requires firmware in the loop. A CPU cannot fetch instructions that way.

XIP presents the flash as read-only memory on an AHB-Lite slave port. A read to the mapped address range is translated automatically into a Quad I/O Read. The CPU issues an ordinary memory read and receives data; it is unaware that a serial flash transaction occurred.

5.2 Address map

Parameter	Value	Description
XIP_BASE	0x8000_0000	Base of the mapped window
XIP_MASK	0xFF80_0000	8 MB window, matching the 64 Mb device

```
in_range = ((HADDR & XIP_MASK) == XIP_BASE);  
flash_addr = {9'd0, HADDR[22:2], 2'b00};    // word aligned
```

5.3 How the master waits

Completion is signalled entirely through AHB wait states. HREADYOUT is held low for the duration of the access; the master stalls, holding its address on the bus, and resumes when HREADYOUT rises.

HREADYOUT	Meaning
1 (at idle)	Ready to accept a transfer. High at rest, or the bus would hang.
0	Access in progress. Master stalled, address held.
1 (one cycle)	HRDATA valid. Transfer completes on this cycle.

5.4 Access latency

Case	Cycles	Notes
Prefetch hit	4	Served from the line buffer, no flash access
Prefetch miss	554	One full Quad I/O Read of the line
Engine busy	+ command length	Waits for the indirect path to finish

Measured values from the directed regression with a 32-byte line.

5.5 Protocol handling

Signal	Handling
HTRANS	Decoded as HTRANS[1]. NONSEQ and SEQ are real transfers; IDLE and BUSY are not and must never start a flash command. Treating BUSY as a transfer would issue a spurious access.
HSIZE	Word (010) only. Sub-word and larger sizes return an ERROR response rather than silently returning the containing word.
HBURST	Not decoded. Each beat is served from its own HADDR, which is correct for every burst type including wrapping bursts, where the address decreases, and unspecified-length INCR, where the beat count is unknown. All burst types are supported.
HREADY	The system ready input. The address phase commits only when it is high; sampling HADDR while it is low would latch an address the master is about to change.
HWDATA	Unused. XIP is read-only; flash is written through the indirect path.

5.6 Error responses

An AHB ERROR response is two cycles: HRESP high with HREADYOUT low, then HRESP high with HREADYOUT high.

Condition	Response
Write to the mapped range	ERROR
HSIZE other than 010	ERROR
Read while xip_en is low	ERROR — quad mode not yet configured
Access exceeds the timeout	ERROR — prevents a stuck flash hanging the bus
Address outside the window	Not claimed; HSEL should not be asserted

5.7 Byte order

The flash returns bytes MSB-first from the lowest address. For a little-endian system the first byte received is placed in HRDATA[7:0].

flash contents at 0x100: 10 11 12 13

HRDATA returned: 0x13121110

5.8 XIP command descriptor

The descriptor presented to the shared engine when XIP owns it. This is the only place in the design where a command is fixed in hardware.

Field	Value	Reason
opcode	0xEB	Quad I/O Read
opcode_line	1-bit	Opcode is single-line on this device
addr_line	4-bit	Address interleaved across IO3..IO0
addr_4byte	0	3-byte addressing
mode_en	0	Enhance byte is folded into the dummy count (5.9)
dummy	6	Device default
data_line	4-bit	Quad data
length	line size	Set by the prefetch buffer, not a constant
data_r_en	1	Read direction
data_w_en	0	XIP never writes
poll_wip_en	0	A read starts no self-timed cycle

5.9 Performance-enhance byte

Quad I/O Read on this device folds an 8-bit performance-enhance byte into the front of its six dummy clocks. The host must continue driving for those clocks rather than releasing the bus immediately, or the device rejects the command.

At quad width, 8 bits occupies two clocks. The engine holds the IO drivers for the first two dummy clocks and releases them for the remaining four. The value driven is 0x00, meaning continuous-read mode off, so the next command still requires a full opcode.

5.10 Bytes per AHB beat

Every AHB read returns exactly one 32-bit word: HSIZE other than word is rejected (5.5), so there is no way for the master to receive more or less than 4 bytes per beat.

The prefetch buffer, underneath that, fetches a whole line from the flash in one Quad I/O Read — 32 bytes by default (XIP_LINE_BYTES, 11.1) — but the CPU still only ever sees 4 of those bytes per transfer. Draining one 32-byte line therefore takes 8 AHB reads: the first misses and pays the full fill latency (554 cycles, 5.4), and the remaining seven are hits served from the line buffer (4 cycles each, 5.4).

This is the counterpart to the read-side limit in 4.5: the indirect path is capped at 255 bytes per command and reads them out one at a time, while XIP has no software-visible cap at all — the CPU simply keeps issuing reads and line fills happen automatically as it crosses line boundaries .

6. Prefetch Buffer

6.1 Purpose

Without a prefetch buffer every AHB beat would issue its own flash command, including sequential beats within a burst. A four-beat burst would cost four full flash accesses.

The buffer fetches a whole line on a miss and serves subsequent reads from local storage. Measured: 554 cycles for a miss against 4 cycles for a hit.

6.2 Organisation

A single line. Not a multi-way cache.

Element	Description
Buffer_Mem	Byte array, LINE_BYTES deep
Tag	Base address of the currently held line
line_valid	Whether the held bytes may be served

Line size is set at elaboration by the LINE_BYTES parameter and supports 16, 32, 64 or 128 bytes. The default is 32. It is not runtime-writable: the line size is a board-level property fixed by the device and the expected access pattern.

6.3 Address decomposition

With a 32-byte line, address 0x114 decomposes as:

```
tag   = HADDR & ~(LINE_BYTES-1) // 0x114 & ~0x1F = 0x100  which line
offset = HADDR & (LINE_BYTES-1) // 0x114 & 0x1F = 0x14   where inside
hit    = line_valid && (tag == Tag);
```

6.4 Fill sequence

1. Request arrives. The full address is captured; the offset is needed at the end to select the word.
2. Tag compare. On a hit the word is selected and returned immediately.
3. On a miss, xip_addr is driven with the line base and xip_length with the line size, then xip_req is raised and held.
4. Wait for xip_done, meaning every byte of the line is in the XIP RX FIFO.
5. Drain the FIFO into the line array. The FIFO presents data two cycles after the read strobe, so the capture is aligned to a delayed strobe.
6. Set the tag and valid flag, then select and return the requested word.

During a fill the AHB master is stalled by HREADYOUT, so no new request can arrive and no fill can be interrupted.

6.5 Non-sequential address during a fill

The design has no speculative prefetch: a fill is only ever started by a demand miss. Because the master is stalled by HREADYOUT for the whole duration of that fill, it is structurally unable to present a new address mid-fill.

The case of a non-sequential address arriving during a line fill is therefore unreachable by construction rather than handled by logic. Speculative next-line prefetch, which would make the case reachable, is described in section 9 as future work.

6.6 Coherency

If firmware erases or programs an address the buffer holds, the cached copy becomes stale. The buffer is invalidated on completion of any indirect command, whether or not that command modified flash contents.

This is deliberately conservative. A status-register read triggers an unnecessary refill. In exchange, the hardware requires no opcode decoding and no address-range comparison, which preserves the property that the controller holds no per-command knowledge.

Finer-grained schemes were considered and rejected: gating on the data-write enable misses erases, which have no data phase, and a single-tag comparison misses multi-line writes, since a page program spans up to 256 bytes and a sector erase 4 KB. Correctly bounding the affected range would require knowing what each opcode does.

An XIP line fill also completes with a core Done. The invalidate signal is qualified with ownership, so a fill cannot invalidate the line it has just loaded.

7. Shared Engine Arbitration

7.1 Principles

- Ownership is exclusive. The two paths can never drive the flash concurrently: there is one engine, one shift register and one set of pins.
- Ownership is latched for the whole transfer. Switching the descriptor mid-command would present a malformed sequence to the flash.
- The indirect path has priority. A continuous stream of XIP fetches must never prevent a configuration or erase command from running.
- Arbitration is performed entirely in SCLK, after both requests have been synchronised, so two asynchronous signals are never compared directly.

7.2 Grant condition

```
if (!xip_owns && !core_busy && !apb_start && xip_req)
    xip_owns <= 1'b1;      // XIP takes the engine
else if (xip_owns && !xip_req)
    xip_owns <= 1'b0;      // released when the fill completes
```

XIP is granted only when the engine is idle AND no indirect command is pending. The `apb_start` term is what gives the indirect path priority.

7.3 Case: APB is running, AHB requests

The AHB read arrives while an indirect command is in flight.

1. The prefetch buffer raises `xip_req` and waits.
2. The arbiter sees `core_busy` high and does not grant. `xip_owns` stays low.
3. The AHB slave keeps `HREADYOUT` low, so the CPU remains stalled.
4. The indirect command completes normally, unaffected. Its data goes to the indirect RX FIFO.
5. `core_busy` falls. The arbiter now grants XIP, the descriptor mux switches, and the line fill runs.
6. The CPU read completes.

The CPU simply waits longer. The indirect path is never interrupted or corrupted. The stall is bounded by the length of the indirect command in progress.

7.4 Case: AHB is running, APB requests

Firmware writes `CONTROL` with `START` while an XIP line fill is in flight.

1. The indirect descriptor crosses to SCLK and `start_Q` asserts.
2. `xip_owns` is already latched high, so the arbiter does not switch. The XIP fill continues to completion.

3. The APB slave holds PREADY low, so the CONTROL write does not complete and firmware is stalled.
4. The fill finishes, the prefetch buffer drops xip_req, and xip_owns clears.
5. The descriptor mux returns to the register descriptor and the indirect command starts.
6. On completion, PREADY is asserted and the firmware write finishes.

The indirect command waits for at most one line fill, approximately 554 cycles. It is never starved, because XIP cannot re-acquire the engine while apb_start is asserted.

7.5 Case: both request simultaneously

The engine is idle and both apb_start and xip_req assert in the same cycle. The grant condition includes !apb_start, so XIP is not granted and the indirect command proceeds. XIP is granted on the following idle period.

7.6 Completion reporting

Both paths complete through the same core Done signal, so the design must distinguish whose command finished. Ownership is latched at the moment the command launches:

```
always @(posedge SCLK)
    if (core_start) xip_owns_at_start <= xip_owns;

    apb_done = qspi_done && !xip_owns_at_start;
    apb_error = qspi_error && !xip_owns_at_start;
```

Without this, an XIP fill completing would release a stalled APB write that had not yet run. The gated signals are pulse-stretched and then synchronised into PCLK for the APB slave.

7.7 Summary

Situation	Indirect path	XIP path	Bound
Indirect running, XIP requests	Completes normally	Waits, CPU stalled	Indirect command length
XIP running, indirect requests	Waits, PREADY low	Completes normally	One line fill (~554 cycles)
Both request together	Proceeds	Waits	Immediate
Both idle	Proceeds on START	Proceeds on miss	—

8. Known Limitations

8.1 SPI mode 0 only; no clock divider

- Flash clock is a fixed inversion of SCLK — no divider, no CPOL/CPHA control.
- Flash always runs at SCLK frequency, mode 0 only.
- Mode 3 devices unsupported, but target device (mode 0) works fine.

8.2 WIP auto-poll is unreliable

- `poll_wip_en` auto-issues RDSR1 until WIP clears, but should not be used.
- Samples at quad width even though RDSR1 is single-line → corrupt status byte.
- No timeout — a stuck WIP bit hangs Busy/PREADY forever.
- Feature is untested by the regression suite.
- Workaround: leave it off, poll RDSR1 from firmware instead.

8.3 Silent FIFO underrun and overflow

- TX/RX enables gate silently on FIFO empty/full — no error flags exist.
- TX overflow: excess bytes pushed before START are silently dropped.
- TX underrun: FSM re-sends stale byte when FIFO runs dry — silent wire corruption.
- RX overflow: not reachable at current 255-byte max, but no margin; RX underrun leaves `data_done` unasserted, hanging the APB bus (needs reset, no timeout).
- Firmware must self-track byte counts as hard constraints: never overfill TX, push all LENGTH bytes before START, never over-read DATA.

8.4 START does not self-clear

- `CONTROL[0]` is a level, not a pulse.
- If firmware doesn't clear it, the FSM immediately relaunches the same command.

8.5 Descriptor registers not write-protected while busy

- Writing `CONTROL/OPCODE/ADDRESS/MODE/DUMMY/LENGTH` mid-command races the descriptor latch (non-deterministic whether it applies).
- DATA writes during transfer are fine and required.

9. Verification Summary

A directed regression runs both paths against the vendor behavioural model of the target device. Bring-up follows the recommended order: RDID, RDSR1, WREN with sector erase and WIP poll, page program with readback, quad enable, indirect Quad I/O Read, and XIP last.

9.1 Command coverage

Command	Opcode	Lines	Result
Write Enable	0x06	1	Verified
Write Disable	0x04	1	Verified
Read Status Register 1	0x05	1	Verified
Read JEDEC ID	0x9F	1	Verified
Normal Read	0x03	1	Verified
Fast Read	0x0B	1	Verified
Page Program	0x02	1	Verified
Sector Erase 4 KB	0x20	1	Verified
Block Erase 64 KB	0xD8	1	Verified
Write Status Register	0x01	1	Verified
Dual Output Read	0x3B	2 data	Verified
Dual I/O Read	0xBB	2 addr + data	Verified
Quad Output Read	0x6B	4 data	Verified
Quad I/O Read	0xEB	4 addr + data	Verified
Quad Page Program	0x38	4 data	Not supported by the model
Read Status Register 2	0x35	1	Not implemented by the device

The final two entries are device and model limitations rather than controller defects: the descriptor for each was issued correctly and the transaction completed on the bus.

9.2 XIP coverage

Area	Result
Read with xip_en low returns ERROR	Verified
Cache miss returns correct data	Verified
Cache hit served without a flash access	Verified, 4 cycles against 554
All words within a line	Verified

Area	Result
Line boundary crossing forces a refill	Verified
INCR4 and INCR8 bursts	Verified
INCR8 crossing a line boundary	Verified
IDLE and BUSY issue no flash command	Verified
Write to the mapped range returns ERROR	Verified
Sub-word read returns ERROR	Verified
Coherency after an indirect write	Verified
HREADYOUT low throughout, rises once	Verified

9.3 Concurrency coverage

Area	Result
Alternating APB and AHB access, six rounds	Verified
XIP read immediately after an indirect command	Verified
Indirect read immediately after an XIP read	Verified
Neither receive FIFO left holding stranded bytes	Verified

10. Integration Notes

10.1 Parameters

Parameter	Default	Legal values	Effect
XIP_LINE_BYTES	32	16, 32, 64, 128	Prefetch line size
XIP_LINE_SHIFT	5	4, 5, 6, 7	Must equal log2 of the line size
XIP_BASE	0x8000_0000	any	Base of the mapped window
XIP_MASK	0xFF80_0000	any	Window size mask

The XIP receive FIFO depth must be at least the prefetch line size: the flash streams the whole line before the buffer begins draining it. A FIFO smaller than the line will silently drop bytes.

10.2 Required initialisation sequence

1. Release reset in all three domains.
2. Issue WREN over the indirect path.
3. Issue WRSR with the QE bit set.

4. Poll RDSR1 until WIP clears.
5. Assert xip_en.

XIP reads issued before xip_en return an AHB ERROR response rather than undefined data, which converts a silent failure into a visible one.

11. Wave Form

Refer The VCD File

https://drive.google.com/file/d/1zM96suw3bR_dWwZBl8DYZWFsLftOgjUY/view?usp=sharing